```
import os, random, warnings
warnings.filterwarnings("ignore")

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import StratifiedKFold, train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, roc_auc_score

import tensorflow as tf
from tensorflow.keras import layers, models, callbacks, regularizers
```

```
SEED = 42
N_SPLITS = 5
EPOCHS = 200
BATCH_SIZE = 128
LEARNING_RATE = 1e-3

def set_seed(seed=42):
    os.environ["PYTHONHASHSEED"] = str(seed)
    random.seed(seed)
    np.random.seed(seed)
    tf.random.set_seed(seed)

set_seed(SEED)
print("TensorFlow version:", tf.__version__)
```

```
TensorFlow version: 2.19.0
```

```
# Load dataset
df = pd.read_csv("https://raw.githubusercontent.com/McahitKutsal/hivcsv/main/HIV7.csv")
print("Dataset shape:", df.shape)
display(df.head())

target_col = "HIV_active"
y = df[target_col].values

# Keep only numeric predictor columns
X_df = df.drop(columns=[target_col]).select_dtypes(include=["float64", "int64", "float32", "int32"]).copy()
print("Numeric feature shape:", X_df.shape)

X = X_df.values.astype("float32")
n_features = X.shape[1]
```

```
Dataset shape: (2885, 4)
```

	Unnamed: 0	smiles	activity	HIV_active
0	0	CNC(=O)Nc1ccncc1	CI	0
1	1	O=C1NC(O)c2cccc(Cl)c2O1	CI	0
2	2	O=S1(=O)C=Cc2ccccc2CC1	CI	0
3	3	Cc1ccc(OS(=O)(=O)c2ccccc2[N+](=O)[O-])cc1	CM	1
4	4	CN(c1ccccc1)S(=O)(=O)c1cccc([N+](=O)[O-])c1	CM	1

```
Numeric feature shape: (2885, 1)
```

```
def build_brnn_model(input_length):
    model = models.Sequential([
        layers.Input(shape=(input_length, 1)),
        layers.Bidirectional(
            layers.SimpleRNN(
                64,
                return_sequences=True,
                dropout=0.2,
                recurrent_dropout=0.0,
                kernel_regularizer=regularizers.l2(1e-4)
            )
        ),
        layers.Bidirectional(
            layers.SimpleRNN(
                32,
                return_sequences=False,
                dropout=0.2,
                recurrent_dropout=0.0,
                kernel_regularizer=regularizers.l2(1e-4)
            )
```

```
            ),
            layers.Dense(64, activation="relu"),
            layers.Dropout(0.3),
            layers.Dense(32, activation="relu"),
            layers.Dropout(0.2),
            layers.Dense(1, activation="sigmoid")
        ])

        model.compile(
            optimizer=tf.keras.optimizers.Adam(learning_rate=LEARNING_RATE),
            loss="binary_crossentropy",
            metrics=["accuracy", tf.keras.metrics.AUC(name="auc")]
        )
        return model
```

```
def evaluate_predictions(y_true, y_prob, threshold=0.5):
    y_pred = (y_prob >= threshold).astype("int32")
    return {
        "accuracy": accuracy_score(y_true, y_pred),
        "precision": precision_score(y_true, y_pred, zero_division=0),
        "recall": recall_score(y_true, y_pred, zero_division=0),
        "f1": f1_score(y_true, y_pred, zero_division=0),
        "roc_auc": roc_auc_score(y_true, y_prob)
    }

def prepare_fold_data(X_train_full, X_test, y_train_full, y_test, random_state):
    X_train, X_val, y_train, y_val = train_test_split(
        X_train_full, y_train_full,
        test_size=0.2,
        random_state=random_state,
        stratify=y_train_full
    )

    scaler = StandardScaler()
    X_train = scaler.fit_transform(X_train)
    X_val = scaler.transform(X_val)
    X_test = scaler.transform(X_test)

    X_train = X_train[..., np.newaxis]
    X_val = X_val[..., np.newaxis]
    X_test = X_test[..., np.newaxis]

    return X_train, X_val, X_test, y_train, y_val, y_test
```

```
def run_brnn_5fold_cv(X, y, n_splits=5, seed=42):
    skf = StratifiedKFold(n_splits=n_splits, shuffle=True, random_state=seed)

    fold_results = []
    histories = []

    for fold, (train_idx, test_idx) in enumerate(skf.split(X, y), start=1):
        print("\n" + "="*28 + f" FOLD {fold} " + "="*28)
        set_seed(seed + fold)

        X_train_full, X_test = X[train_idx], X[test_idx]
        y_train_full, y_test = y[train_idx], y[test_idx]

        X_train, X_val, X_test, y_train, y_val, y_test = prepare_fold_data(
            X_train_full, X_test, y_train_full, y_test, random_state=seed + fold
        )

        model = build_brnn_model(input_length=X_train.shape[1])

        early_stop = callbacks.EarlyStopping(
            monitor="val_loss",
            patience=10,
            restore_best_weights=True,
            verbose=1
        )

        reduce_lr = callbacks.ReduceLROnPlateau(
            monitor="val_loss",
            factor=0.5,
            patience=5,
            min_lr=1e-5,
            verbose=1
        )

        history = model.fit(
            X_train, y_train,
            validation_data=(X_val, y_val),
            epochs=EPOCHS,
            batch_size=BATCH_SIZE,
            callbacks=[early_stop, reduce_lr],
            verbose=0
        )
```

```
        histories.append(history.history)

        y_val_prob = model.predict(X_val, verbose=0).ravel()
        y_test_prob = model.predict(X_test, verbose=0).ravel()

        val_metrics = evaluate_predictions(y_val, y_val_prob, threshold=0.5)
        test_metrics = evaluate_predictions(y_test, y_test_prob, threshold=0.5)

        row = {
            "fold": fold,
            "val_accuracy": val_metrics["accuracy"],
            "val_precision": val_metrics["precision"],
            "val_recall": val_metrics["recall"],
            "val_f1": val_metrics["f1"],
            "val_roc_auc": val_metrics["roc_auc"],
            "test_accuracy": test_metrics["accuracy"],
            "test_precision": test_metrics["precision"],
            "test_recall": test_metrics["recall"],
            "test_f1": test_metrics["f1"],
            "test_roc_auc": test_metrics["roc_auc"],
            "epochs_trained": len(history.history["loss"])
        }
        fold_results.append(row)

        print("Validation:",
              {k.replace("val_", ""): round(v, 4) for k, v in row.items() if k.startswith("val_")})
        print("Test:",
              {k.replace("test_", ""): round(v, 4) for k, v in row.items() if k.startswith("test_")})
        print("Epochs trained:", row["epochs_trained"])

    results_df = pd.DataFrame(fold_results)

    metric_cols = ["test_accuracy", "test_precision", "test_recall", "test_f1", "test_roc_auc"]
    summary_rows = []
    for col in metric_cols:
        mean_val = results_df[col].mean()
        std_val = results_df[col].std(ddof=1)
        var_val = results_df[col].var(ddof=1)
        summary_rows.append({
            "Metric": col.replace("test_", "").upper(),
            "Mean": mean_val,
            "Std": std_val,
            "Variance": var_val,
            "Formatted": f"{mean_val:.4f} ± {std_val:.4f}"
        })

    summary_df = pd.DataFrame(summary_rows)
    return results_df, summary_df, histories
```

```
results_df, summary_df, histories = run_brnn_5fold_cv(X, y, n_splits=N_SPLITS, seed=SEED)

print("\nFold bazlı sonuçlar:")
display(results_df)

print("\nÖzet sonuçlar:")
display(summary_df)

results_df.to_csv("BRNN_fold_results.csv", index=False)
summary_df.to_csv("BRNN_summary_results.csv", index=False)

print("Saved: BRNN_fold_results.csv")
print("Saved: BRNN_summary_results.csv")
```

```
def plot_average_history(histories):
    max_len = max(len(h["loss"]) for h in histories)

    train_loss_mat, val_loss_mat = [], []
    train_acc_mat, val_acc_mat = [], []

    for h in histories:
        tl = np.array(h["loss"], dtype=float)
        vl = np.array(h["val_loss"], dtype=float)
        ta = np.array(h["accuracy"], dtype=float)
```

```
        va = np.array(h["val_accuracy"], dtype=float)

        def pad(arr, length):
            if len(arr) < length:
                arr = np.pad(arr, (0, length-len(arr)), mode="edge")
            return arr

        train_loss_mat.append(pad(tl, max_len))
        val_loss_mat.append(pad(vl, max_len))
        train_acc_mat.append(pad(ta, max_len))
        val_acc_mat.append(pad(va, max_len))

    avg_train_loss = np.mean(train_loss_mat, axis=0)
    avg_val_loss = np.mean(val_loss_mat, axis=0)
    avg_train_acc = np.mean(train_acc_mat, axis=0)
    avg_val_acc = np.mean(val_acc_mat, axis=0)

    epochs = np.arange(1, max_len + 1)

    plt.figure(figsize=(8,5))
    plt.plot(epochs, avg_train_acc, label="Train Accuracy")
    plt.plot(epochs, avg_val_acc, label="Validation Accuracy")
    plt.xlabel("Epoch")
    plt.ylabel("Accuracy")
    plt.title("BRNN Average Accuracy Across 5 Folds")
    plt.legend()
    plt.grid(True)
    plt.show()

    plt.figure(figsize=(8,5))
    plt.plot(epochs, avg_train_loss, label="Train Loss")
    plt.plot(epochs, avg_val_loss, label="Validation Loss")
    plt.xlabel("Epoch")
    plt.ylabel("Loss")
    plt.title("BRNN Average Loss Across 5 Folds")
    plt.legend()
    plt.grid(True)
    plt.show()
```

Epochs trained: 23

```
plot_average_history(histories)
```

```
Epoch 11: ReduceLROnPlateau reducing learning rate to 0.0005000000237487257.

Epoch 16: ReduceLROnPlateau reducing learning rate to 0.0002500000118743628.
Epoch 16: early stopping
Restoring model weights from the end of the best epoch: 6.
Validation: {'accuracy': 0.5108, 'precision': 0.5108, 'recall': 1.0, 'f1': 0.6762, 'roc_auc': np.float64(0.4977)}
Test: {'accuracy': 0.5113, 'precision': 0.5113, 'recall': 1.0, 'f1': 0.6766, 'roc_auc': np.float64(0.4832)}
Epochs trained: 16

============================ FOLD 5 ============================

Epoch 29: ReduceLROnPlateau reducing learning rate to 0.0005000000237487257.

Epoch 35: ReduceLROnPlateau reducing learning rate to 0.0002500000118743628.

Epoch 40: ReduceLROnPlateau reducing learning rate to 0.0001250000059371814.

Epoch 49: ReduceLROnPlateau reducing learning rate to 6.25000029685907e-05.

Epoch 64: ReduceLROnPlateau reducing learning rate to 3.125000148429535e-05.

Epoch 69: ReduceLROnPlateau reducing learning rate to 1.5625000742147677e-05.

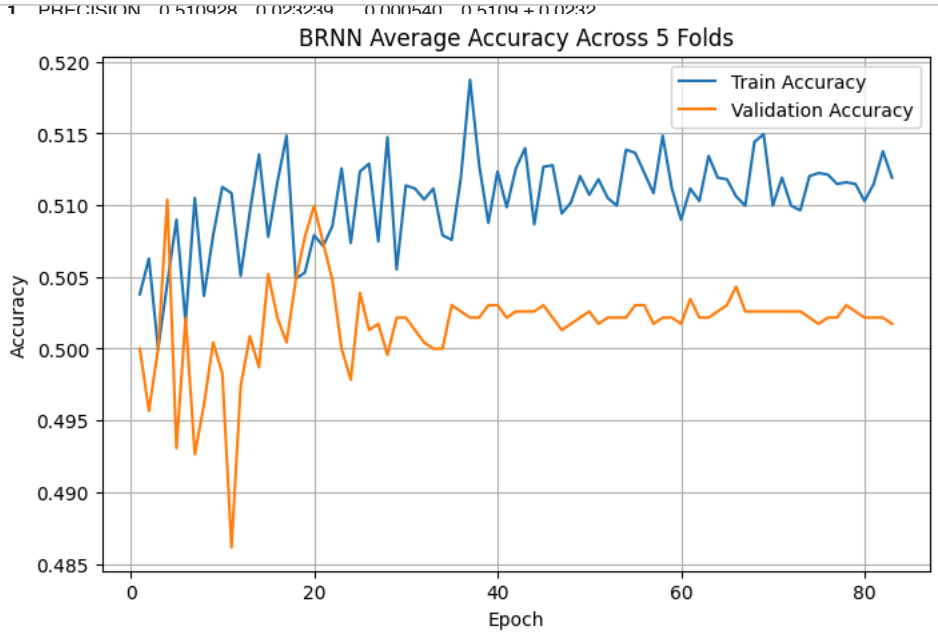
Epoch 74: ReduceLROnPlateau reducing learning rate to 1e-05.
Epoch 83: early stopping
Restoring model weights from the end of the best epoch: 73.
Validation: {'accuracy': 0.5087, 'precision': 0.5123, 'recall': 0.7924, 'f1': 0.6223, 'roc_auc': np.float64(0.499
Test: {'accuracy': 0.5147, 'precision': 0.5172, 'recall': 0.7627, 'f1': 0.6164, 'roc_auc': np.float64(0.5079)}
Epochs trained: 83

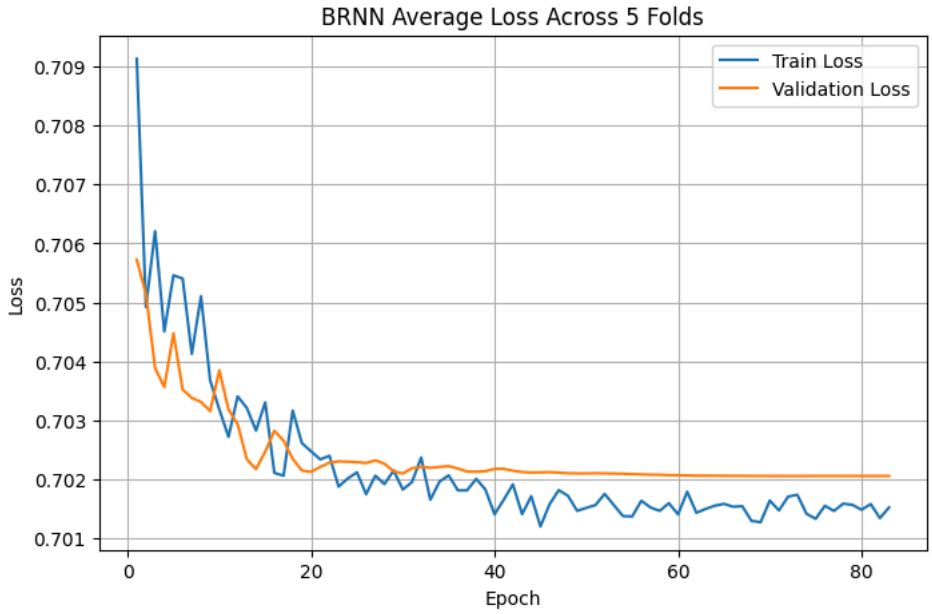
Fold bazlı sonuçlar:
```

	fold	val_accuracy	val_precision	val_recall	val_f1	val_roc_auc	test_accuracy	test_precision	test_recall
0	1	0.521645	0.530612	0.550847	0.540541	0.523699	0.462738	0.475758	0.534014
1	2	0.500000	0.508711	0.618644	0.558317	0.492791	0.502600	0.509749	0.622449
2	3	0.521645	0.530612	0.550847	0.540541	0.542504	0.533795	0.540625	0.586441
3	4	0.510823	0.510823	1.000000	0.676218	0.497694	0.511265	0.511265	1.000000
4	5	0.508658	0.512329	0.792373	0.622296	0.499344	0.514731	0.517241	0.762712

Özet sonuçlar:

	Metric	Mean	Std	Variance	Formatted
0	ACCURACY	0.505026	0.026244	0.000689	0.5050 ± 0.0262






```
def plot_average_history(histories):
    import numpy as np
    import matplotlib.pyplot as plt

    max_len = max(len(h["loss"]) for h in histories)

    train_loss_mat, val_loss_mat = [], []
    train_acc_mat, val_acc_mat = [], []

    for h in histories:
        tl = np.array(h["loss"], dtype=float)
        vl = np.array(h["val_loss"], dtype=float)
        ta = np.array(h["accuracy"], dtype=float)
        va = np.array(h["val_accuracy"], dtype=float)

        def pad(arr, length):
            if len(arr) < length:
                arr = np.pad(arr, (0, length - len(arr)), mode="edge")
            return arr

        train_loss_mat.append(pad(tl, max_len))
        val_loss_mat.append(pad(vl, max_len))
        train_acc_mat.append(pad(ta, max_len))
        val_acc_mat.append(pad(va, max_len))

    avg_train_loss = np.mean(train_loss_mat, axis=0)
    avg_val_loss = np.mean(val_loss_mat, axis=0)
    avg_train_acc = np.mean(train_acc_mat, axis=0)
    avg_val_acc = np.mean(val_acc_mat, axis=0)

    epochs = np.arange(1, max_len + 1)

    # Fold bazlı stop epoch
    stop_epochs = [len(h["loss"]) for h in histories]
```

```
# %75 persentil early stop epoch
early_stop_epoch = int(np.percentile(stop_epochs, 75))
early_stop_epoch = max(1, min(early_stop_epoch, max_len))

# İlgili nokta değerleri
early_acc_y = avg_val_acc[early_stop_epoch - 1]
early_loss_y = avg_val_loss[early_stop_epoch - 1]

# ===== Accuracy =====
plt.figure(figsize=(8,5))
plt.plot(epochs, avg_train_acc, linewidth=2, label="Train Accuracy")
plt.plot(epochs, avg_val_acc, linewidth=2, label="Validation Accuracy")

plt.axvline(x=early_stop_epoch, color="red", linestyle="--", linewidth=2)
plt.scatter(early_stop_epoch, early_acc_y, color="red", s=80, zorder=5)
plt.text(
    early_stop_epoch + 1,
    early_acc_y,
    f"Early Stop Epoch = {early_stop_epoch}",
    color="red",
    fontsize=10
)

plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("BRNN Average Accuracy Across 5 Folds")
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

# ===== Loss =====
plt.figure(figsize=(8,5))
plt.plot(epochs, avg_train_loss, linewidth=2, label="Train Loss")
plt.plot(epochs, avg_val_loss, linewidth=2, label="Validation Loss")

plt.axvline(x=early_stop_epoch, color="red", linestyle="--", linewidth=2)
plt.scatter(early_stop_epoch, early_loss_y, color="red", s=80, zorder=5)
plt.text(
    early_stop_epoch + 1,
    early_loss_y,
    f"Early Stop Epoch = {early_stop_epoch}",
    color="red",
    fontsize=10
)

plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("BRNN Average Loss Across 5 Folds")
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

print("Fold-based stop epochs:", stop_epochs)
print("75th Percentile Early Stop Epoch:", early_stop_epoch)
```

```
def plot_average_history(histories):
    import numpy as np
    import matplotlib.pyplot as plt

    max_len = max(len(h["loss"]) for h in histories)

    train_loss_mat, val_loss_mat = [], []
    train_acc_mat, val_acc_mat = [], []

    for h in histories:
        tl = np.array(h["loss"], dtype=float)
        vl = np.array(h["val_loss"], dtype=float)
        ta = np.array(h["accuracy"], dtype=float)
        va = np.array(h["val_accuracy"], dtype=float)

        def pad(arr, length):
            if len(arr) < length:
                arr = np.pad(arr, (0, length - len(arr)), mode="edge")
            return arr

        train_loss_mat.append(pad(tl, max_len))
        val_loss_mat.append(pad(vl, max_len))
        train_acc_mat.append(pad(ta, max_len))
        val_acc_mat.append(pad(va, max_len))

    avg_train_loss = np.mean(train_loss_mat, axis=0)
    avg_val_loss = np.mean(val_loss_mat, axis=0)
    avg_train_acc = np.mean(train_acc_mat, axis=0)
    avg_val_acc = np.mean(val_acc_mat, axis=0)

    epochs = np.arange(1, max_len + 1)

    stop_epochs = [len(h["loss"]) for h in histories]
```